Day 95: Scaling-Laws Intuition — a Real Experiment Varying Model Size and Data

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 14, Day 5 | Complete study guide

Watch: The Chinchilla Correction: Unpacking LLM Scaling Laws (Latent AI) | The AI Scaling Problem, Explained (Vivek Gupta)

Script: `day95_scaling_laws.py` · every number in this guide is copied by program from `day95_run_output.txt`, the real output of that script.

Seven runs of Day 90's GPT with the same 3,000-step budget. Making the model bigger lowers the held-out loss along a smooth curve: a power law with exponent 0.251 fits all four sizes within 0.004 bits. Giving it less data does something else: with a fixed budget the small-data runs overfit badly (the smallest is seen 195.9 times and its held-out loss rises to 4.888), and no power law describes the data curve well, not even with each run's best checkpoint (worst miss 0.094 bits). Four points per curve make any exponent rough, and the lesson measures how rough.

0. Where today fits

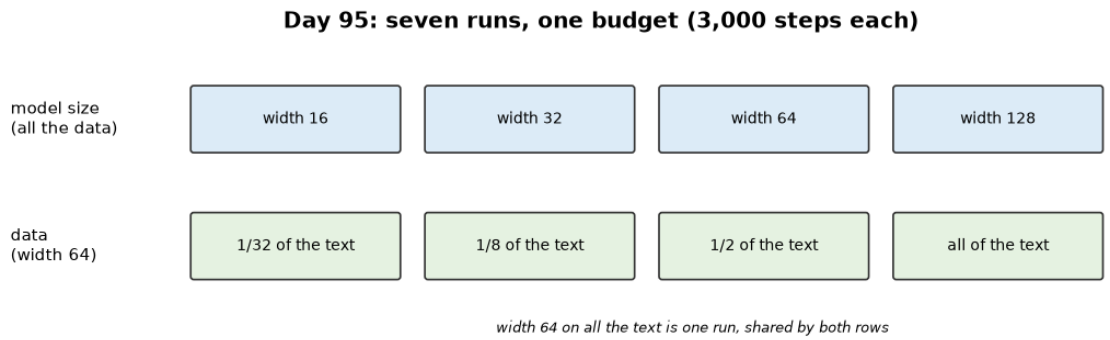
DEFINITION — start here, no prior knowledge assumed

A **scaling law** describes how a model's held-out loss changes as one resource grows: the number of weights N, the amount of training data D, or the compute. Large studies (Kaplan et al., 2020; Hoffmann et al., 2022) found losses close to a **power law**, $L = c + a\,N^{-\alpha}$: each doubling of N removes a fixed fraction of the loss above a floor c that no size removes.

REAL-WORLD SCENARIO

Hiring more people for a job: each extra person helps less than the one before, and some delays never shrink however many you hire.

SYSTEM DIAGRAM



Seven runs, one budget: four sizes, four amounts of data.

Today's part for Day 97

`day95_exercise.py` is the weight count by formula and the power-law fit, checked on made-up losses with known parameters, with and without noise.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. How many weights

DEFINITION — start here, no prior knowledge assumed

N counts every number training changes. For Day 90's GPT it has a closed form: the token and position tables, the blocks (four d × d projections, the feed-forward layers, two LayerNorms each), the output layer.

REAL-WORLD SCENARIO

Counting the bricks in a house from its plan rather than by taking it apart.

MATHEMATICS

$$N = V\,d + T\,d + L\,(4d^2 + 2\,d\,f + f + d + 4d) + d\,V + V \quad (V = 65 \text{ characters}, T = 64, L = 2 \text{ blocks}, f = 4d)$$

ACTUAL OUTPUT

```
 width d  by formula  in the model  in the blocks
      16        9601          9601           6432
      32       31425         31425          25152
      64      111937        111937          99456
     128      420417        420417         395520
Actual output from this lesson's run.
```

What it means

The formula matches the model at every width. Doubling the width roughly quadruples the blocks (99456 to 395520); at width 16 the tables are a third of the model (9601 weights, 6432 in the blocks).

WHY CHOOSE THIS?

Lets the size axis be chosen exactly.

WHY NOT THIS? (TRADEOFFS)

Counts weights, not compute: a longer context costs more without more weights.

2. Seven runs, one budget

DEFINITION — start here, no prior knowledge assumed

Four widths on all the training text, and four amounts of text at width 64 (the full-text width-64 run counts in both). Every run: 3,000 steps of 32 windows of 64 characters, Day 92's warm-up and cosine schedule with peak 0.003, seed 0; held-out bits per character with full context at seven checks.

REAL-WORLD SCENARIO

Seven students, the same number of study hours: some with bigger notebooks, some with fewer books.

SPECIFICATION

Model	Day 90's GPT, 2 blocks, 4 heads, context 64, feed-forward 4d.
Budget	3,000 steps × 32 × 64 = 6.1 million characters read by every run.
Measured	Day 92's full-context held-out bits, and the same on training text at the end.

Why this test

A fixed budget is the honest version of 'what if I had a bigger model?' when compute is the limit, which on one CPU it always is.

IMPLEMENTATION

```
def run(job):
    name, d, frac = job
    _, _, vocab, tr, va = d92.data()
    tr = tr[:int(len(tr) * frac)]
    with threadpool_limits(1, "blas"):
        rng = np.random.RandomState(0)
        P = d90.init_gpt(rng, len(vocab), d=d, ff=4 * d, layers=LAYERS, t=T)
        state, curve, t0 = {"t": 0, "m": {}, "v": {}}, [], time.perf_counter()
        for step in range(1, STEPS + 1):
            o = rng.randint(0, len(tr) - T - 1, B)
            x = np.stack([tr[i:i + T] for i in o]); y = np.stack([tr[i + 1:i + T + 1] for i in o])
            _, G = d90.gpt_loss_grads(x, y, P, h=H)
            d90.adam_step(P, G, state, d92.lr_schedule(step - 1, PEAK, total=STEPS))
            if step in CHECKS:
                curve.append((step, d92.eval_bits(P, va, T, H), d92.eval_bits(P, tr[:len(va)], T,
                    H)))
        n = sum(a.size for _, a in d90.w12.leaves(P))
    return dict(name=name, d=d, frac=frac, chars=len(tr), weights=n, curve=np.array(curve),
        minutes=(time.perf_counter() - t0) / 60)
```

ACTUAL OUTPUT

```
run                           weights    chars    250    500   1000   1500   2000   2500   3000  train text   min
width 16                         9601  1003854  3.758  3.461  3.285  3.180  3.120  3.085  3.072       3.002     8
width 32                        31425  1003854  3.593  3.238  2.988  2.882  2.834  2.795  2.777       2.592     9
width 64                       111937  1003854  3.384  3.026  2.801  2.653  2.603  2.552  2.534       2.279    14
width 128                      420417  1003854  3.208  2.897  2.675  2.543  2.451  2.388  2.361       2.088    27
width 64, 0.03125 of the text  111937    31370  3.627  3.548  3.683  3.971  4.314  4.642  4.888       1.158    14
width 64, 0.125 of the text    111937   125481  3.611  3.373  3.314  3.263  3.281  3.284  3.277       1.983    14
width 64, 0.5 of the text      111937   501927  3.477  3.131  3.039  2.950  2.928  2.908  2.895       2.234    15
(columns: held-out bits per character after that many steps; train text: the same measured on training text at the end)
Actual output from this lesson's run.
```

What it means

Bigger is better at every check, and the gap does not close: at step 3,000 the widths give 3.072, 2.777, 2.534 and 2.361 bits, and width 128 is still falling. Less text is worse, but not smoothly: on 1/32 of the text the training loss falls to 1.158 while the held-out loss climbs to 4.888. The seven runs took 27 minutes together.

WHY CHOOSE THIS?

Every point is a real, complete training run.

WHY NOT THIS? (TRADEOFFS)

One seed per run; differences of a few hundredths could be noise.

3. Fitting power laws, and how much four points can say

DEFINITION — start here, no prior knowledge assumed

$L = c + a\,x^{-\alpha}$ fitted by trying 400 floors c and fitting a straight line to log(L – c) against log x for each. Then each point is left out in turn and the fit repeated: how far alpha moves shows how well four points pin it down.

REAL-WORLD SCENARIO

Drawing a trend line through four dots, then checking how much it swings if one dot is wrong.

MATHEMATICS

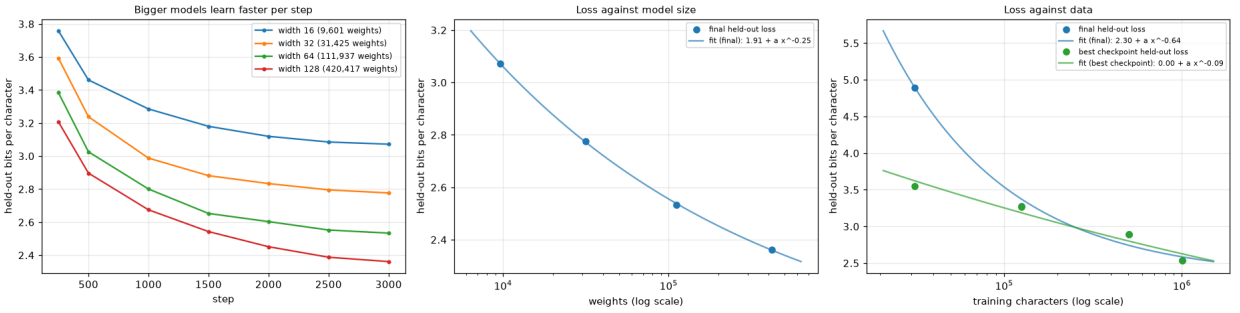
$$\log(L - c) = \log a - \alpha \log x; \quad c \text{ chosen to minimise } \Sigma(c + a\,x^{-\alpha} - L)^2$$

IMPLEMENTATION

```
def fit_power_law(N, L):
    N, L = np.asarray(N, float), np.asarray(L, float)
    best = None
    for c in np.linspace(0, L.min() * 0.999, 400):
        slope, icpt = np.polyfit(np.log(N), np.log(L - c), 1)
        err = np.sum((c + np.exp(icpt) * N ** slope - L) ** 2)
        if best is None or err < best[0]:
            best = (err, c, np.exp(icpt), -slope)
    return best[1:]
```

ACTUAL OUTPUT

```
model size: x = 9601, 31425, 111937, 420417; held-out bits = 3.072, 2.777, 2.534, 2.361
  fit: c = 1.910, alpha = 0.251; leaving one point out, alpha = 0.30, 0.27, 0.24, 0.21
  largest gap between the fit and a point: 0.004 bits
data: x = 31370, 125481, 501927, 1003854; held-out bits = 4.888, 3.277, 2.895, 2.534
  fit: c = 2.303, alpha = 0.640; leaving one point out, alpha = 0.12, 0.19, 0.70, 1.04
  largest gap between the fit and a point: 0.152 bits
data, best checkpoint: x = 31370, 125481, 501927, 1003854; held-out bits = 3.548, 3.263, 2.895, 2.534
  fit: c = 0.000, alpha = 0.093; leaving one point out, alpha = 0.12, 0.09, 0.10, 0.07
  largest gap between the fit and a point: 0.094 bits
Actual output from this lesson's run.
```



Left: held-out loss during training, by model size. Middle: final loss against weights with the fit. Right: loss against training text, final and best checkpoint, with their fits.

What it means

Model size follows a power law closely: alpha 0.251, every point within 0.004 bits, and leaving a point out moves alpha only between 0.21 and 0.30. The fitted floor c (1.910) is an extrapolation far below every measured point; it should not be read as a real limit. **Data** does not: with final losses the fit misses by 0.152 bits and alpha jumps from 0.12 to 1.04 as points are left out. With each run's best checkpoint the curve is tamer but still bent (worst miss 0.094), and the best floor is 0, the edge of the range allowed: the four points cannot place a floor at all.

WHY CHOOSE THIS?

A compact, testable summary of the size curve.

WHY NOT THIS? (TRADEOFFS)

Four points, one seed, one budget: the exponent is a description, not a law.

4. Where data runs out

DEFINITION — start here, no prior knowledge assumed

The width-64 runs again: how many times each training character was seen, and the gap between training-text and held-out loss.

REAL-WORLD SCENARIO

Rereading one short book two hundred times: you learn the book, not the language.

ACTUAL OUTPUT

```
training text    characters passes over it  train text  held-out    gap
0.03125               31370          195.9       1.158     4.888  3.730
0.125                125481           49.0       1.983     3.277  1.294
0.5                  501927           12.2       2.234     2.895  0.661
1                   1003854            6.1       2.279     2.534  0.255
step with the lowest held-out loss: 0.03125: 500, 0.125: 1500, 0.5: 3000, 1: 3000
Actual output from this lesson's run.
```

What it means

On 1/32 of the text each character is seen 195.9 times; the gap reaches 3.730 bits and the held-out loss was lowest at step 500, then got worse for 2,500 steps. On all the text (6.1 passes) the gap is 0.255 and the loss is still falling at the end. With little data, a fixed budget measures overfitting, not the value of data; the large studies mostly trained on each token about once.

WHY CHOOSE THIS?

Shows the overfitting directly, through the gap.

WHY NOT THIS? (TRADEOFFS)

Early stopping or more regularisation would change the small-data numbers; not tested.

Interview preparation — technical questions

Q: How do you fit L = c + a N^-alpha?

A: Try floors c on a grid; for each, a straight line through log(L – c) against log N; keep the best c.

Q: How do you check a fitted exponent?

A: Leave each point out and refit; here alpha moved 0.21-0.30 for size, 0.12-1.04 for data.

Q: Why measure the training-text loss too?

A: The gap to held-out loss shows overfitting: 3.730 bits on 1/32 of the data.

Q: What does 'passes over the data' mean?

A: Characters read in training divided by characters available: 195.9 for the smallest run.

Q: Why the same steps for every run?

A: A fixed compute budget: the question is what the same work buys at each size.

Interview preparation — theoretical questions

Q: Why did model size follow a power law here but data did not?

A: Every size run saw plenty of data; the small-data runs repeated their text many times and overfit.

Q: Is the fitted floor c a real limit?

A: No: 1.910 is far below every measured point; four points do not justify the extrapolation.

Q: What did Hoffmann et al. (Chinchilla) change?

A: They scaled data with model size for a given compute, instead of mostly growing the model.

Q: Why do bigger models learn faster per step?

A: Measured here, not explained: width 128 led at every check. More capacity per update is the usual explanation.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> Even at a tiny scale, held-out loss falls smoothly with model size, close to a power law (alpha 0.251, within 0.004 bits). Data is different under a fixed budget: small datasets are repeated until the model memorises them, and the curve stops being a clean law. Four points give a rough exponent, and leaving one out shows how rough.

- Size: 3.072, 2.777, 2.534, 2.361 bits for 9,601 to 420,417 weights; alpha 0.251 (0.21-0.30 leaving one out).
- Data, final losses: worst miss 0.152 bits, alpha 0.12-1.04 leaving one out.
- Data, best checkpoints: worst miss 0.094 bits; the floor sits at the edge of the range.
- 1/32 of the text: seen 195.9 times, gap 3.730 bits, best at step 500.

Suggested watch order, if starting from zero

- 1. The AI Scaling Problem, Explained (Vivek Gupta) — the story from Kaplan et al. to Chinchilla.
- 2. The Chinchilla Correction: Unpacking LLM Scaling Laws (Latent AI) — the two scaling laws and what the correction changed.

Day 96 preview

- **Fine-tuning.** The mini-GPT trained on all of Shakespeare, then fine-tuned on a narrower real task with little data: how much does starting from a trained model help?